Commercial Queries - SQL Analysis

```
/* ============================================================
   SQL Commercial Analysis Project - Retail Dataset
   Author: Cristóbal Elton
   Description: Business-focused SQL queries using products,
   customers, and sales data from a simulated retail company.
   ============================================================ */

-- ============================================================
-- SECTION 1: Basic Exploration
-- ============================================================

-- 1. Top 10 best-selling products by total quantity
SELECT
    p.nombre AS product_name,
    SUM(v.cantidad) AS total_units_sold
FROM ventas v
JOIN productos p ON v.producto_id = p.id
GROUP BY p.nombre
ORDER BY total_units_sold DESC
LIMIT 10;

-- 2. Total annual revenue
SELECT
    EXTRACT(YEAR FROM fecha) AS year,
    SUM(total) AS total_revenue
FROM ventas
GROUP BY year
ORDER BY year;

-- 3. Average product price by subcategory
SELECT
    subcategoria,
    ROUND(AVG(precio)) AS avg_price
FROM productos
GROUP BY subcategoria
ORDER BY avg_price DESC;

-- ============================================================
-- SECTION 2: Intermediate Analysis
-- ============================================================

-- 4. Customers who made more than 3 purchases
SELECT
    c.id, c.nombre,
    COUNT(v.id) AS purchase_count
FROM clientes c
JOIN ventas v ON v.cliente_id = c.id
GROUP BY c.id, c.nombre
HAVING COUNT(v.id) > 3
ORDER BY purchase_count DESC;
```

```sql
-- 5. Top brand revenue
SELECT
    p.marca,
    SUM(v.total) AS total_revenue
FROM ventas v
JOIN productos p ON v.producto_id = p.id
GROUP BY p.marca
ORDER BY total_revenue DESC;

-- 6. Top-selling product per year
WITH yearly_sales AS (
    SELECT
        p.nombre,
        EXTRACT(YEAR FROM v.fecha) AS year,
        SUM(v.total) AS total_sales
    FROM ventas v
    JOIN productos p ON v.producto_id = p.id
    GROUP BY p.nombre, year
),
ranked_products AS (
    SELECT *,
           RANK() OVER (PARTITION BY year ORDER BY total_sales DESC) AS rank
    FROM yearly_sales
)
SELECT * FROM ranked_products
WHERE rank = 1;

-- ============================================================
-- SECTION 3: Advanced Queries (CTEs, Window Functions)
-- ============================================================

-- 7. Customer spending segmentation
SELECT
    id,
    nombre,
    total_spent,
    CASE
        WHEN total_spent >= 1000000 THEN 'High Value'
        WHEN total_spent >= 500000 THEN 'Medium Value'
        ELSE 'Low Value'
    END AS segment
FROM (
    SELECT
        c.id,
        c.nombre,
        SUM(v.total) AS total_spent
    FROM clientes c
    JOIN ventas v ON v.cliente_id = c.id
    GROUP BY c.id, c.nombre
) AS sub;
```

```sql
-- 8. Running total of sales per product
SELECT
    p.nombre,
    v.fecha,
    SUM(v.total) OVER (PARTITION BY v.producto_id ORDER BY v.fecha) AS cumulative_sales
FROM ventas v
JOIN productos p ON v.producto_id = p.id
ORDER BY p.nombre, v.fecha;

-- 9. Days between purchases for each customer
SELECT
    cliente_id,
    fecha,
    LAG(fecha) OVER (PARTITION BY cliente_id ORDER BY fecha) AS previous_purchase,
    fecha - LAG(fecha) OVER (PARTITION BY cliente_id ORDER BY fecha) AS days_between
FROM ventas;

-- 10. First purchase date per customer
SELECT
    cliente_id,
    MIN(fecha) AS first_purchase
FROM ventas
GROUP BY cliente_id;

-- 11. Monthly sales summary
SELECT
    DATE_TRUNC('month', fecha) AS month,
    COUNT(*) AS total_transactions,
    SUM(total) AS total_revenue
FROM ventas
GROUP BY month
ORDER BY month;

-- 12. Rank products by sales within each category
SELECT
    nombre,
    categoria,
    SUM(total) AS revenue,
    RANK() OVER (PARTITION BY categoria ORDER BY SUM(total) DESC) AS rank_within_category
FROM ventas v
JOIN productos p ON v.producto_id = p.id
GROUP BY nombre, categoria
ORDER BY categoria, rank_within_category;

-- ============================================================
-- SECTION 4: Cohort, Trends & Comparative Analysis
-- ============================================================

-- 13. First and last purchase per customer
SELECT
    cliente_id,
```

```
    MIN(fecha) AS first_purchase,
    MAX(fecha) AS last_purchase
FROM ventas
GROUP BY cliente_id;

-- 14. Customers who haven't purchased in the last 12 months
SELECT
    c.id, c.nombre
FROM clientes c
LEFT JOIN ventas v ON v.cliente_id = c.id
GROUP BY c.id, c.nombre
HAVING MAX(v.fecha) < CURRENT_DATE - INTERVAL '12 months';

-- 15. Cohort: number of clients registered per month and when they made their first purchase
WITH cohort_base AS (
    SELECT
        id AS cliente_id,
        DATE_TRUNC('month', fecha_registro) AS cohort_month
    FROM clientes
),
first_orders AS (
    SELECT
        cliente_id,
        MIN(fecha) AS first_order_date
    FROM ventas
    GROUP BY cliente_id
)
SELECT
    cb.cohort_month,
    DATE_TRUNC('month', fo.first_order_date) AS first_purchase_month,
    COUNT(*) AS num_clients
FROM cohort_base cb
JOIN first_orders fo ON cb.cliente_id = fo.cliente_id
GROUP BY cb.cohort_month, first_purchase_month
ORDER BY cohort_month, first_purchase_month;

-- 16. Sales per quarter for each product
SELECT
    p.nombre,
    DATE_TRUNC('quarter', v.fecha) AS quarter,
    SUM(v.total) AS total_sales
FROM ventas v
JOIN productos p ON p.id = v.producto_id
GROUP BY p.nombre, quarter
ORDER BY p.nombre, quarter;

-- 17. Year-over-year sales comparison by category
WITH ventas_con_categoria AS (
    SELECT
        p.categoria,
        v.total,
```

```
        EXTRACT(YEAR FROM v.fecha) AS year
    FROM ventas v
    JOIN productos p ON p.id = v.producto_id
)
SELECT
    categoria,
    year,
    SUM(total) AS total_revenue
FROM ventas_con_categoria
GROUP BY categoria, year
ORDER BY categoria, year;

-- 18. Customers with highest average ticket
SELECT
    c.id,
    c.nombre,
    ROUND(AVG(v.total), 0) AS avg_ticket
FROM ventas v
JOIN clientes c ON v.cliente_id = c.id
GROUP BY c.id, c.nombre
ORDER BY avg_ticket DESC
LIMIT 10;

-- 19. Monthly revenue and units sold
SELECT
    DATE_TRUNC('month', fecha) AS month,
    SUM(total) AS revenue,
    SUM(cantidad) AS units_sold
FROM ventas
GROUP BY month
ORDER BY month;

-- 20. Most active customers per year
WITH ranked_customers AS (
    SELECT
        c.id AS cliente_id,
        c.nombre,
        EXTRACT(YEAR FROM v.fecha) AS year,
        COUNT(*) AS orders,
        RANK() OVER (PARTITION BY EXTRACT(YEAR FROM v.fecha) ORDER BY COUNT(*) DESC) AS rank
    FROM ventas v
    JOIN clientes c ON c.id = v.cliente_id
    GROUP BY c.id, c.nombre, year
)
SELECT * FROM ranked_customers
WHERE rank = 1;

-- 21. Products that were never sold
SELECT
    p.id, p.nombre
FROM productos p
```

```
LEFT JOIN ventas v ON v.producto_id = p.id
WHERE v.id IS NULL;

-- 22. Total number of active vs inactive products
SELECT
    activo,
    COUNT(*) AS product_count
FROM productos
GROUP BY activo;

-- 23. Running total and moving average of monthly revenue
WITH monthly_sales AS (
    SELECT
        DATE_TRUNC('month', fecha) AS month,
        SUM(total) AS revenue
    FROM ventas
    GROUP BY month
)
SELECT
    month,
    revenue,
    SUM(revenue) OVER (ORDER BY month) AS running_total,
      ROUND(AVG(revenue) OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 0) AS
moving_avg_3months
FROM monthly_sales
ORDER BY month;

-- 24. Repeat purchase rate (clients who bought more than once)
WITH client_orders AS (
    SELECT cliente_id, COUNT(*) AS total_orders
    FROM ventas
    GROUP BY cliente_id
)
SELECT
    COUNT(*) FILTER (WHERE total_orders > 1)::float / COUNT(*) AS repeat_purchase_rate
FROM client_orders;

-- 25. Products with most revenue variability (stddev)
SELECT
    p.nombre,
    STDDEV(v.total) AS revenue_stddev
FROM ventas v
JOIN productos p ON v.producto_id = p.id
GROUP BY p.nombre
ORDER BY revenue_stddev DESC
LIMIT 10;
```